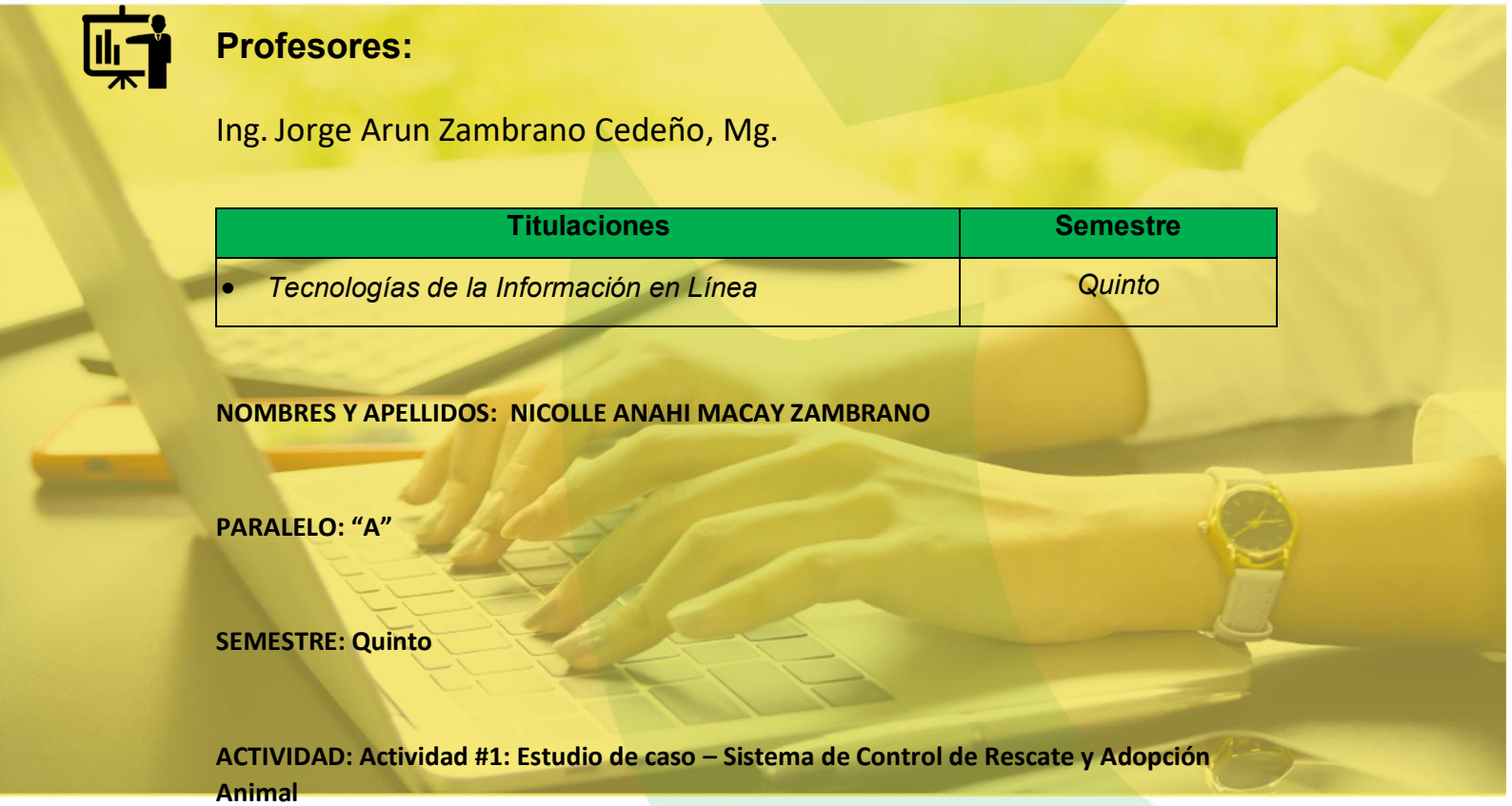
UTM online

Universidad Técnica de Manabí desde 1952

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: NICOLLE ANAHI MACAY ZAMBRANO

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD: Actividad #1: Estudio de caso – Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027

Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de caso – Sistema de Control de Rescate y Adopción Animal

1. Introducción y objetivo

Los refugios animales reciben cada día reportes de perros, gatos y fauna silvestre que requieren atención urgente. Cuando estos casos se gestionan con llamadas, mensajes sueltos u hojas de cálculo, se pierde información, se duplica trabajo y no existe trazabilidad sobre quién atendió cada animal ni en qué estado se encuentra.

Este documento presenta el diseño de la arquitectura lógica de un **Sistema de Control de Rescate y Adopción Animal** que permite reportar ingresos de animales, asignar voluntarios o veterinarios responsables y gestionar el ciclo de vida del caso (Rescatado, En Tratamiento, En Adopción, Adoptado). El diseño aplica modelado UML, cuatro patrones de diseño de software (Singleton, Observer, Factory Method y MVC) y el flujo de trabajo Gitflow sobre un repositorio público en GitHub.

Objetivo: diseñar una arquitectura escalable, reutilizable y mantenible, justificando técnicamente cada patrón y evidenciando el control de versiones del diseño.

2. Análisis del problema

2.1 Actores principales

Actor	Descripción	Interacción principal
Ciudadano / Reportante	Persona que encuentra un animal en situación de riesgo y lo reporta.	Reporta el rescate y consulta el estado de su caso.
Voluntario / Veterinario	Personal operativo del refugio que atiende a los animales.	Recibe notificaciones, atiende el caso, registra tratamientos y actualiza el estado.
Administrador del Refugio	Responsable de la gestión general del refugio.	Asigna responsables, formaliza adopciones, gestiona usuarios y genera reportes.

2.2 Requerimientos funcionales

Código	Requerimiento
RF01	Crear un caso de rescate con los datos del animal, la ubicación del hallazgo y el reportante.
RF02	Categorizar la especie (Perro, Gato, Fauna Silvestre) y generar el expediente correspondiente.
RF03	Asignar un voluntario o veterinario encargado del caso.
RF04	Gestionar los estados del proceso: Rescatado, En Tratamiento, En Adopción, Adoptado/Liberado.
RF05	Notificar automáticamente a los voluntarios y veterinarios cuando ingresa un nuevo rescate.
RF06	Registrar tratamientos médicos y observaciones en el expediente.
RF07	Registrar la adopción y sus datos (adoptante, fecha, contrato).
RF08	Consultar el historial y el estado actual de un caso.

2.3 Requerimientos no funcionales

Código	Atributo	Descripción
RNF01	Escalabilidad	Poder agregar nuevas especies o canales de notificación sin modificar el código existente.
RNF02	Mantenibilidad	Bajo acoplamiento y alta cohesión mediante capas (MVC) y principios SOLID.
RNF03	Trazabilidad de versiones	Todo avance del diseño queda registrado en GitHub mediante Gitflow.
RNF04	Disponibilidad	Las notificaciones deben generarse de inmediato al registrar un caso.
RNF05	Usabilidad	Interfaz sencilla para que un ciudadano pueda reportar en pocos pasos.

3. Diagrama de casos de uso

El diagrama muestra la interacción de los tres actores con el sistema. Los casos «include» indican comportamiento obligatorio (por ejemplo, al reportar un animal

siempre se categoriza la especie y se crea el expediente) y «extend» comportamiento opcional (registrar tratamiento al actualizar el estado).

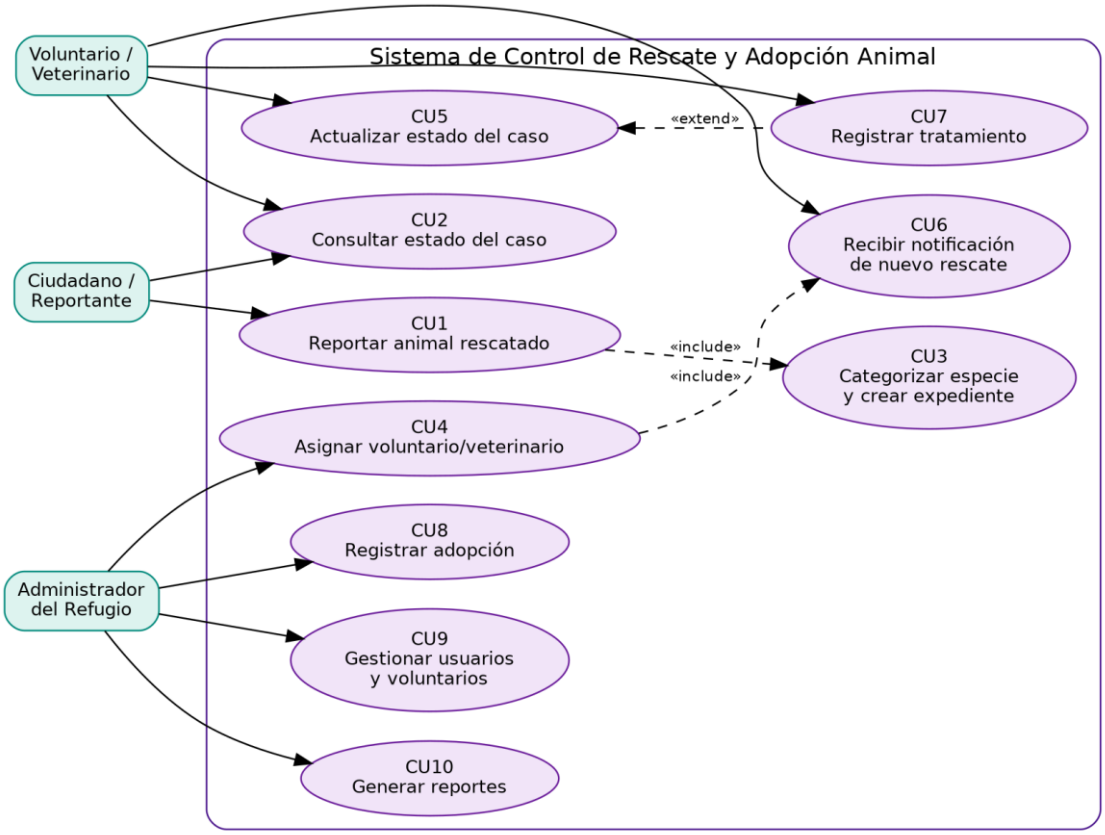


Figura 1. Diagrama de casos de uso.

4. Diagrama de clases UML y principios SOLID

La Figura 2 presenta el diagrama de clases completo. Las clases en verde corresponden al patrón Observer, las rojas a Factory Method, y *GestorCasos* implementa Singleton.

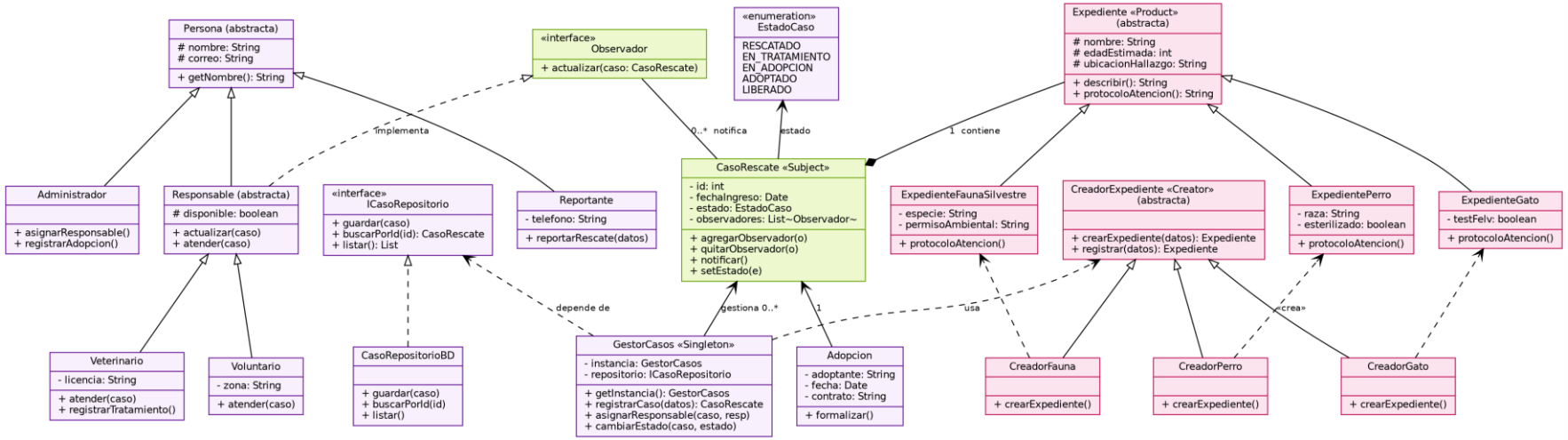


Figura 2. Diagrama de clases UML del sistema.

4.1 Aplicación de los principios SOLID

Principio	Aplicación en el diseño
S - Responsabilidad única	*CasoRescate* maneja el estado del caso; *Expediente* los datos del animal; *GestorCasos* la coordinación; *CasoRepositorioBD* la persistencia.
O - Abierto/Cerrado	Para soportar una nueva especie (p. ej. Ave) se crea *ExpedienteAve* y *CreadorAve* sin modificar las clases existentes.
L - Sustitución de Liskov	*ExpedientePerro*, *ExpedienteGato* y *ExpedienteFaunaSilvestre* sustituyen a *Expediente*; *Voluntario* y *Veterinario* a *Responsable*.
I - Segregación de interfaces	La interfaz *Observador* expone un único método (*actualizar*) y *ICasoRepositorio* solo las operaciones de persistencia.
D - Inversión de dependencias	*GestorCasos* depende de la abstracción *ICasoRepositorio*, no de una base de datos concreta.

5. Patrones de diseño aplicados

Se aplican cuatro patrones: tres que resuelven problemas concretos del dominio (Singleton, Observer, Factory Method) y uno estructural (MVC) que organiza todo el sistema.

5.1 Singleton (creacional): GestorCasos

Propósito: garantizar una única instancia de una clase y ofrecer un punto de acceso global a ella.

Problema que resuelve: si cada pantalla o controlador creara su propio gestor, existirían varias conexiones a datos y listas de casos inconsistentes (dos voluntarios podrían quedar asignados al mismo caso). *GestorCasos* es el motor central de gestión: mantiene la conexión al repositorio y coordina el registro, la asignación y el cambio de estado. Su constructor es privado y se accede mediante *getInstancia()*.

Justificación técnica: asegura consistencia de los datos y ahorro de recursos. Debe implementarse de forma segura ante hilos (inicialización diferida con bloqueo) para el acceso concurrente de varios usuarios.

5.2 Observer (comportamiento): notificaciones a voluntarios

Propósito: definir una dependencia uno-a-muchos de modo que, cuando un objeto cambia de estado, todos sus dependientes sean notificados automáticamente.

Problema que resuelve: cuando ingresa un nuevo rescate, los voluntarios y veterinarios deben enterarse de inmediato, sin tener que consultar el sistema

constantemente. *CasoRescate* actúa como Subject y mantiene una lista de Observadores (*Voluntario*, *Veterinario*); al llamar *setEstado()* ejecuta *notificar()*, que invoca *actualizar(caso)* en cada uno.

Justificación técnica: desacopla al caso de sus receptores (es posible agregar un nuevo observador, como un envío de correo o SMS, sin tocar *CasoRescate*) y es la base de la programación reactiva.

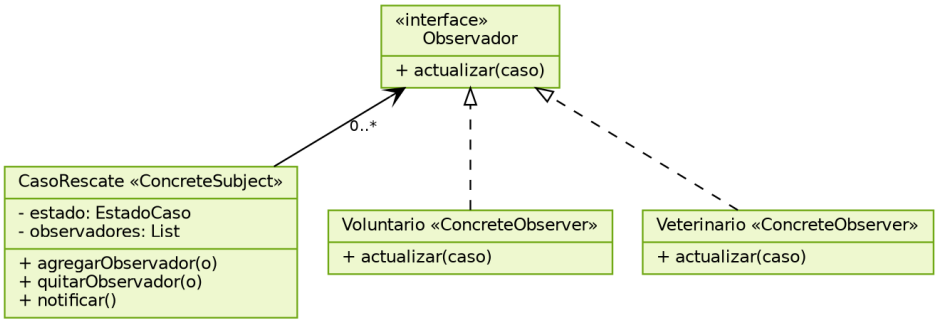


Figura 3. Diagrama de clases del patrón Observer.

5.3 Factory Method (creacional): expedientes por especie

Propósito: definir una interfaz para crear objetos, dejando que las subclases decidan qué clase concreta instanciar.

Problema que resuelve: cada especie requiere un expediente distinto (el perro registra raza y esterilización; el gato, prueba de FeLV; la fauna silvestre, permiso ambiental). Sin el patrón, el sistema tendría una cadena de *if/else* o *switch* por especie que habría que modificar cada vez que se agregue una nueva. Con *CreadorExpediente* y sus subclases (*CreadorPerro*, *CreadorGato*, *CreadorFauna*), cada una implementa *crearExpediente()* y devuelve el producto adecuado.

Justificación técnica: cumple el principio Abierto/Cerrado y centraliza la lógica de creación.

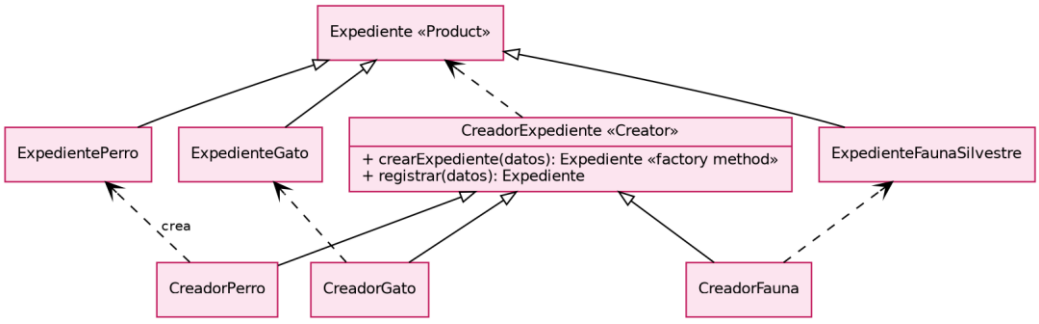


Figura 4. Diagrama de clases del patrón Factory Method.

5.4 MVC (estructural): organización del sistema

Propósito: separar la lógica de negocio (Modelo), la interfaz (Vista) y la coordinación entre ambas (Controlador).

Problema que resuelve: evita mezclar formularios con reglas de negocio. La *Vista* (formularios y paneles) captura eventos del usuario; el *Controlador* (*CasoController*, *AdopcionController*) los interpreta y llama al Modelo; el *Modelo* (*GestorCasos*, expedientes, casos, repositorio) contiene la lógica. Esto permite cambiar la interfaz (web, móvil) sin tocar las reglas del refugio.

Resumen de patrones:

Patrón	Tipo	Clase(s) del sistema	Beneficio
Singleton	Creacional	GestorCasos	Instancia y conexión única
Factory Method	Creacional	CreadorExpediente y subclases	Creación de expedientes sin condicionales
Observer	Comportamiento	CasoRescate, Observador, Voluntario, Veterinario	Notificaciones automáticas
MVC	Estructural / arquitectónico	Vistas, Controladores, Modelo	Separación de responsabilidades

6. Diagramas de secuencia

Secuencia 1 - Reporte de un rescate. Muestra cómo el controlador obtiene la instancia única del gestor, el gestor delega la creación del expediente al Factory Method correspondiente a la especie, crea el caso y lo persiste.

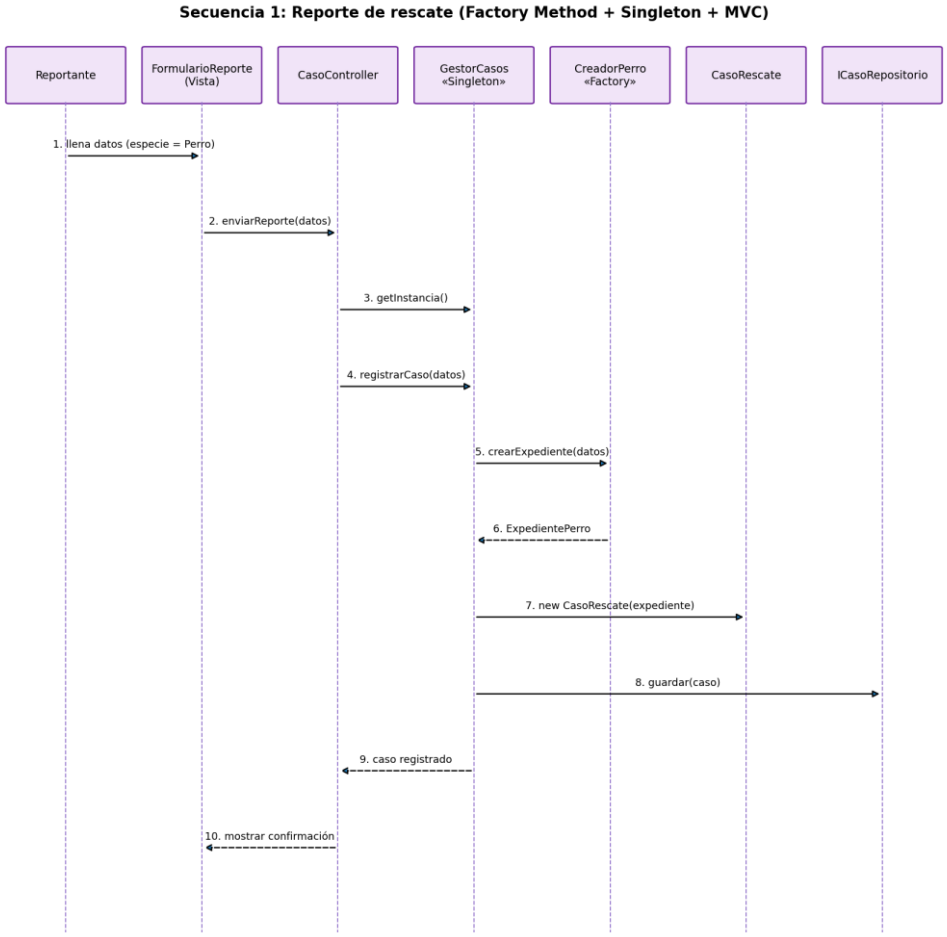


Figura 5. Diagrama de secuencia: reporte de rescate (Factory Method + Singleton + MVC).

Secuencia 2 - Notificación automática. Cuando el caso cambia de estado, el Subject recorre su lista de observadores y notifica a cada voluntario y veterinario suscrito.

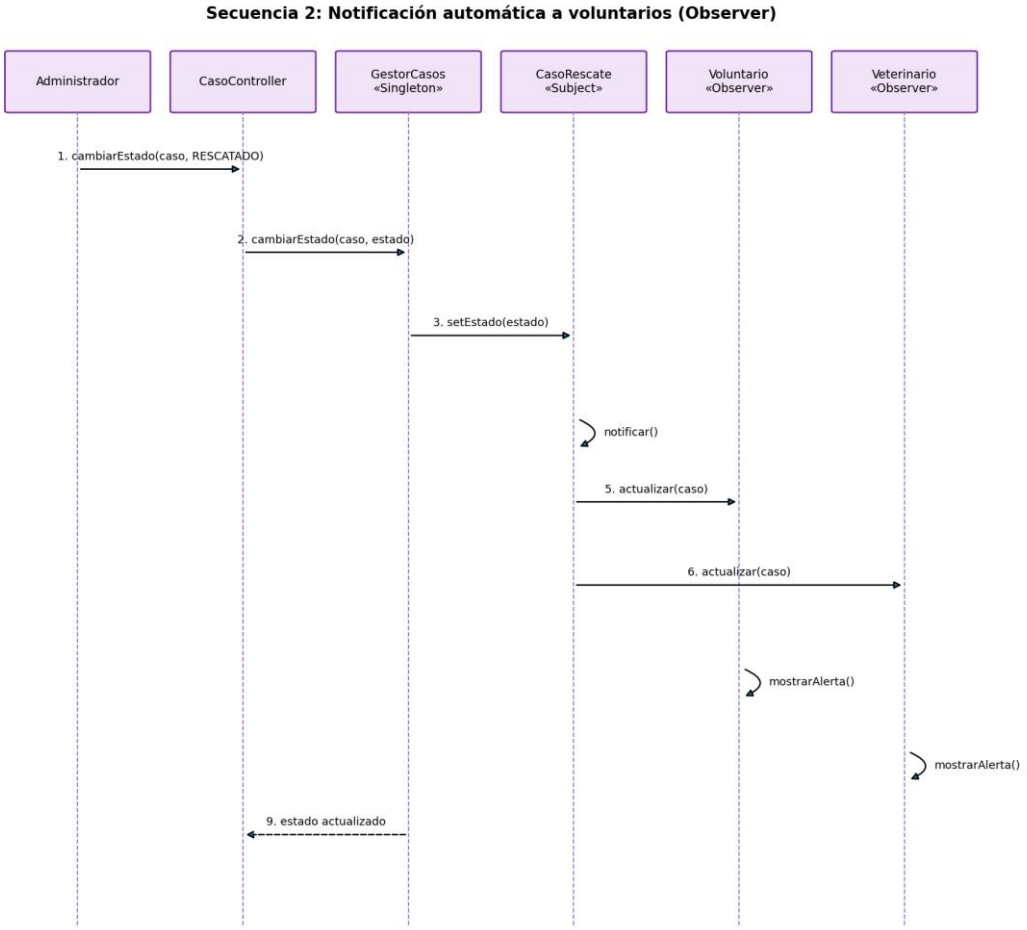


Figura 6. Diagrama de secuencia: notificación a voluntarios (Observer).

7. Arquitectura general del sistema

La arquitectura se organiza en tres capas MVC. La vista interactúa solo con los controladores; estos usan el Gestor de Casos (Singleton), que crea expedientes mediante la fábrica, administra los casos observables y accede a los datos a través de

la interfaz del repositorio. Todo el diseño y el código se versionan en GitHub con el flujo Gitflow.

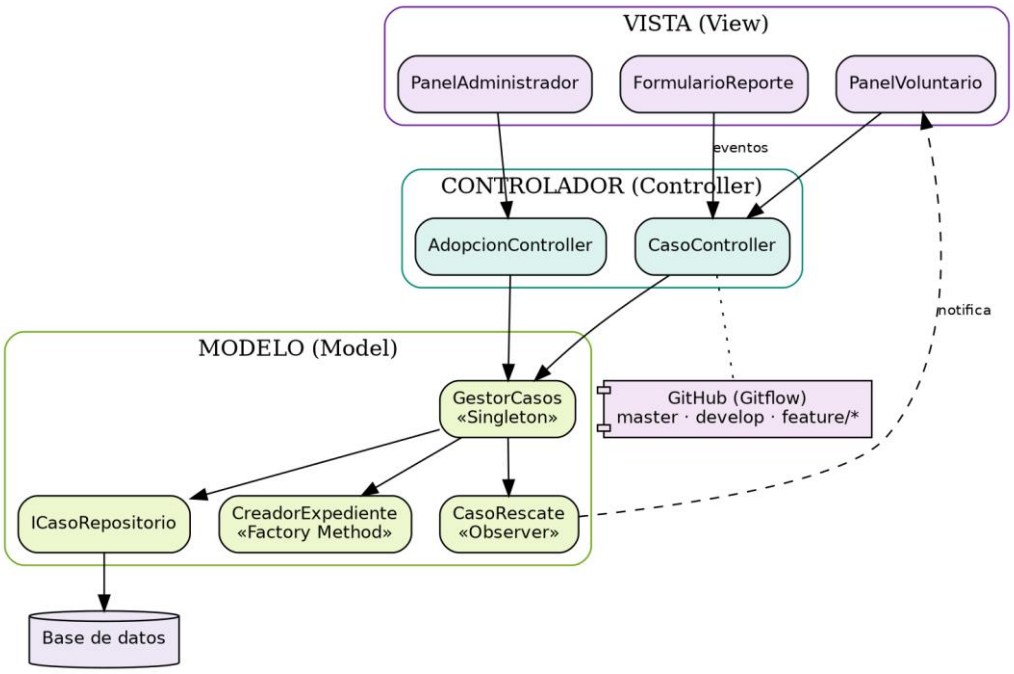


Figura 7. Arquitectura general e integración con el flujo de versiones.

8. Gitflow: justificación y evidencias

Enlace al repositorio: https://github.com/nmacay9233-dev/rescate-adopcion-animal

Justificación: Gitflow define ramas con roles claros, lo que da orden y trazabilidad a un proyecto. Se eligió porque (1) la rama *main* (equivalente a *master*) siempre contiene versiones estables y entregables; (2) *develop* integra el trabajo en curso; (3) cada avance del diseño (casos de uso, clases, patrones, secuencia) se desarrolla aislado en una rama *feature/* y se integra mediante una solicitud de extracción (pull request); y (4) el historial permite saber quién cambió qué y revertir errores.

Rama	Función	Ramas de este proyecto
main (master)	Versiones estables y entregables	Versión 1.0 (entrega final)
develop	Integración continua del diseño	Rama de integración
feature/*	Un avance del diseño por rama	feature/casos-uso, feature/diagrama-clases, feature/patrones, feature/diagramas-secuencia
release/*	Preparación de la entrega	release/1.0

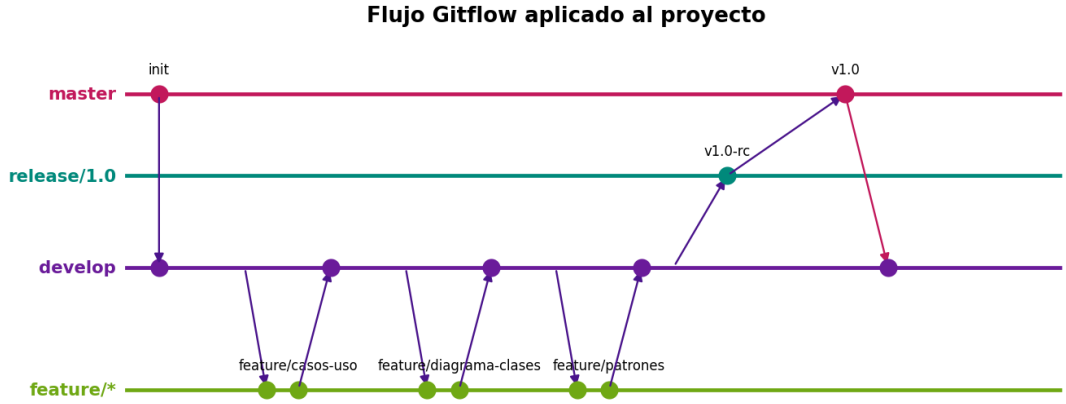


Figura 8. Esquema del flujo Gitflow del proyecto.

Las evidencias del historial de ramas y commits del repositorio (Figuras 9 y 10) se presentan en la sección Evidencias de este documento.

9. Conclusiones

El diseño propuesto separa responsabilidades y permite que el sistema crezca sin reescribir lo existente: nuevas especies se agregan con nuevas fábricas, nuevos canales de aviso con nuevos observadores y nuevas interfaces sin tocar el modelo. El Singleton garantiza consistencia en la gestión de casos, el Observer asegura que los voluntarios se enteren a tiempo de cada rescate, y MVC mantiene el código ordenado.

Gitflow aportó una disciplina de trabajo profesional: cada entregable del diseño quedó registrado en una rama propia, se integró mediante pull requests y se publicó como versión estable en *main* a través de *release/1.0*, lo que garantiza la trazabilidad solicitada en los requerimientos no funcionales.

Evidencias

Enlace al repositorio: https://github.com/nmacay9233-dev/rescate-adopcion-animal

Evidencias del historial de ramas y commits

La Figura 9 muestra el gráfico de ramas del repositorio en GitHub (Insights > Network), donde se observan las ramas feature, release, develop y main. La Figura 10 muestra el historial de commits, con los avances de cada rama y las cinco fusiones (pull requests #1 al #5).

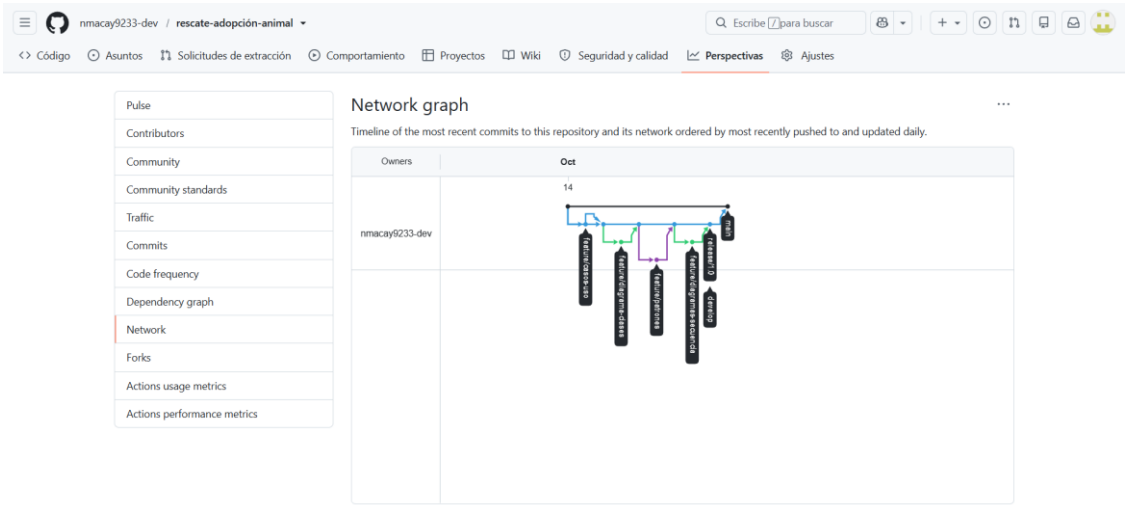


Figura 9. Gráfico de ramas del repositorio (Network graph).

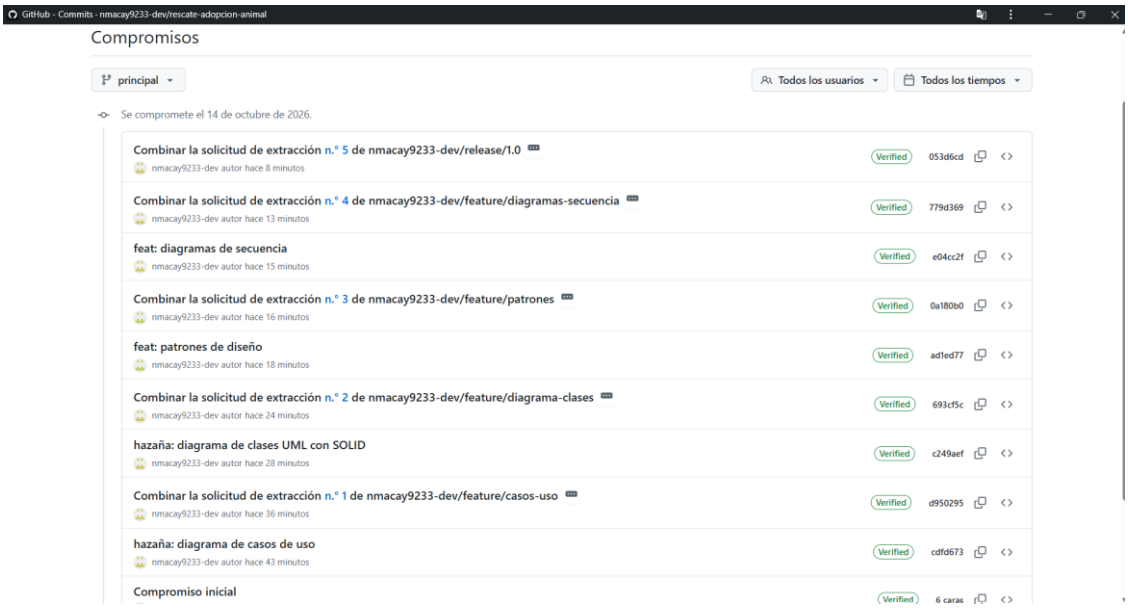


Figura 10. Historial de commits del repositorio.

Bibliografía

Driessen, V. (2010). A successful Git branching model. nvie.com.

Fowler, M. (2003). *UML Distilled* (3.ª ed.). Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.